

Week 10 demo — fallback captures

Trace a new feature: reserve a bike for 15 minutes

Captured with Claude Code, Sonnet 5 (low effort), September
2026

When to use this deck

Switch here if the live AI fails. These are real answers to the demo's own prompts; walk the same defect catalogue along the chain, requirement to test and back.

Prompt #1

“For a new feature — a rider can reserve a bike for 15 minutes before pickup — give me the requirement, the use case, the class changes, the sequence, and a test.”

Al's answer — requirements and use case I

ID	Requirement
R1	A rider can reserve one available bike at a station for 15 minutes.
R2	While reserved, the bike can't be unlocked or reserved by anyone else.
R3	A rider can hold at most one active reservation.
R4	The reservation ends when the rider picks up the bike (converted to a trip), cancels it, or 15 minutes pass (expired).
R5	On expiry or cancellation, the bike returns to Available.
R6	The rider sees the remaining time and is notified shortly before expiry.
R7	Only the reserving rider can unlock the reserved bike.

Al's answer — requirements and use case II

Use Case: Reserve a Bike [...]

Precondition: The rider is logged in, has no active reservation or trip, and has a valid payment method. [...]

4. The system shows a confirmation with a countdown.
5. The rider arrives and unlocks the bike. The reservation becomes Converted and a trip starts.
 - ▶ 5a. The rider cancels. The reservation becomes Cancelled and the bike becomes Available.
 - ▶ 5b. 15 minutes pass. The reservation becomes Expired and the bike becomes Available.
 - ▶ 5c. A different rider tries to unlock the bike. The system denies it.

AI's answer — class changes I

```
Bike
+ status: BikeStatus          // add RESERVED to AVAILABLE, IN_USE, MAINTENANCE
+ reserve(): void             // AVAILABLE -> RESERVED, else throws
+ release(): void             // RESERVED -> AVAILABLE
```

```
Reservation                      (NEW)
- id, rider: Rider, bike: Bike
- createdAt, expiresAt: Instant
- state: ReservationState      // ACTIVE, CONVERTED, CANCELLED, EXPIRED
+ isExpired(now): boolean
+ convert(): Trip
+ cancel(): void
+ expire(): void
```

```
ReservationService              (NEW)
- reservations: ReservationRepository
- clock: Clock                  // injected so tests control time
- HOLD = Duration.ofMinutes(15)
+ reserve(riderId, bikeId): Reservation
+ cancel(reservationId): void
+ expireDue(): int              // called by scheduler
```

AI's answer — class changes II

```
ReservationExpiryJob                (NEW)  // runs every ~30s, calls expireDue()
```

```
Rider
```

```
  + activeReservation(): Optional<Reservation>
```

```
TripService (changed)
```

```
  + startTrip(riderId, bikeId)
```

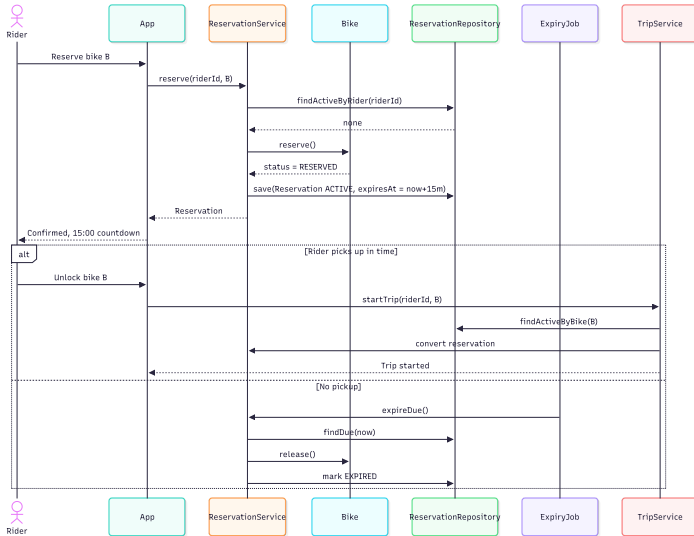
```
    // if the bike is RESERVED, allow it only for the holder,
```

```
    // then call reservation.convert()
```

```
ReservationRepository              (NEW)
```

```
  + findActiveByRider(riderId), findActiveByBike(bikeId), findDue(now)
```

AI's answer — the sequence



Al's answer — the tests

```
class ReservationServiceTest {
    MutableClock clock = new MutableClock(Instant.parse("2026-09-29T10:00:00Z"));
    InMemoryReservationRepository repo = new InMemoryReservationRepository();
    ReservationService service = new ReservationService(repo, clock);

    @Test
    void reservationExpiresAfter15MinutesAndFreesBike() {
        Rider rider = new Rider("r1");
        Bike bike = new Bike("b1"); // AVAILABLE

        Reservation res = service.reserve(rider, bike);

        assertEquals(BikeStatus.RESERVED, bike.getStatus());
        assertEquals(clock.instant().plus(Duration.ofMinutes(15)), res.getExpiresAt());

        clock.advance(Duration.ofMinutes(14).plusSeconds(59));
        assertEquals(0, service.expireDue()); // not yet
        assertEquals(BikeStatus.RESERVED, bike.getStatus());

        clock.advance(Duration.ofSeconds(1)); // exactly 15:00
        assertEquals(1, service.expireDue());
        assertEquals(ReservationState.EXPIRED, res.getState());
        assertEquals(BikeStatus.AVAILABLE, bike.getStatus());
    }
}
```

Al's answer — the tests (continued)

```
@Test
void cannotReserveAlreadyReservedBike() {
    Bike bike = new Bike("b1");
    service.reserve(new Rider("r1"), bike);
    assertThrows(BikeUnavailableException.class,
        () -> service.reserve(new Rider("r2"), bike));
}

@Test
void riderCannotHoldTwoReservations() {
    Rider rider = new Rider("r1");
    service.reserve(rider, new Bike("b1"));
    assertThrows(ActiveReservationExistsException.class,
        () -> service.reserve(rider, new Bike("b2")));
}

@Test
void onlyHolderCanUnlockReservedBike() {
    Bike bike = new Bike("b1");
    service.reserve(new Rider("r1"), bike);
    assertThrows(NotReservationHolderException.class,
        () -> tripService.startTrip(new Rider("r2"), bike));
}
```

The first test is the acceptance test for R1, R4 and R5. It checks the 14:59 and 15:00 boundary. The other three cover R2, R3 and R7.

Prompt #2 (same session)

“Make the layers consistent: every operation the sequence or the test calls must be declared in the class changes, including Rider; every clause of the requirement — expiry, ‘unavailable to other riders’, cancel — must have a test; only the rider who reserved may unlock; drop anything no requirement asked for. Then list the trace links: requirement -> use case -> class -> sequence -> test.”

Revised — requirements and class changes I

Dropped because no requirement asked for them: the one-reservation-per-rider rule, the countdown and notification, the payment and eligibility checks, the “nearby bikes” suggestion, `Rider.activeReservation()`, `ReservationExpiryJob`, and `findActiveByRider`. [...]

ID	Requirement
R1	A rider can reserve an available bike; the reservation lasts 15 minutes.
R2	While reserved, the bike is unavailable to other riders (they can't reserve it).
R3	After 15 minutes without pickup the reservation expires and the bike is available again.
R4	The rider can cancel the reservation; the bike is available again.
R5	Only the rider who reserved the bike may unlock it. Unlocking is the pickup and ends the reservation.

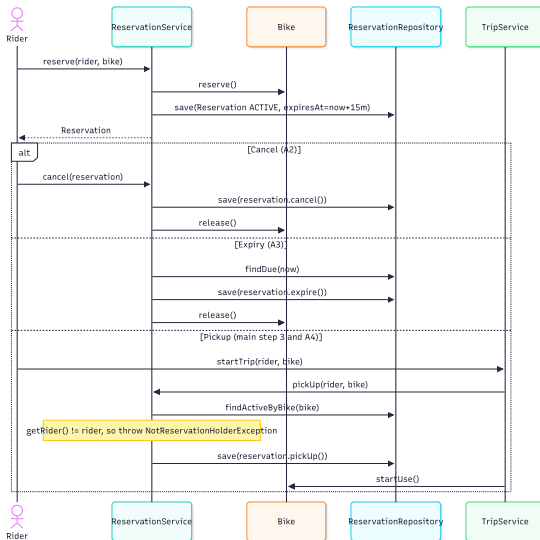
Revised — requirements and class changes III

```
ReservationRepository          (new)
+ save(Reservation): void
+ findActiveByBike(bike): Optional<Reservation>
+ findDue(now: Instant): List<Reservation>
```

```
BikeUnavailableException, NotReservationHolderException  (new)
```

Test support: MutableClock (with advance(Duration)) and InMemoryReservationRepository.

Revised — the sequence



Revised — the trace links I

Req	Use case	Classes and operations	Sequence	Tests
R1	UC1 steps 1-2	ReservationService.reserve, Bike.reserve, Reservation.getExpiresAt, ReservationRepository.save	Reserve	reserveHoldsBikeFor15Minutes
R2	UC1 A1	Bike.reserve (throws BikeUnavailableException), ReservationService.reserve	Reserve (throw before save)	otherRiderCannotReserveHeldBike
R3	UC1 A3	ReservationService.expireDue, Reservation.isDue and expire, Bike.release, ReservationRepository.findDue	Expiry	expiresAtExactly15MinutesAndFreesBike, and the last assertion of holderUnlockEndsReservation
R4	UC1 A2	ReservationService.cancel, Reservation.cancel, Bike.release	Cancel	cancelFreesBike
R5	UC1 steps 3-4 and A4	TripService.startTrip, ReservationService.pickUp, Reservation.getRider, Rider.equals, Reservation.pickUp, Bike.startUse, ReservationRepository.findActiveByBike, NotReservationHolderException	Pickup	otherRiderCannotUnlock, holderUnlockEndsReservation

Revised — the trace links II

Coverage check

- ▶ Every requirement has at least one test.
- ▶ Every operation called in the tests or sequences appears in section 3. [...]
- ▶ **Open question:** what happens if the holder unlocks after 15:00 but before `expireDue()` has run. The requirements don't say. As written, the pickup would succeed. If that's wrong, add an R3 clause and a test.

Make-it-fail reserve (fresh session, after prompt #1)

“Now add loyalty points, surge pricing, and fraud detection to this feature.”

This changes one earlier assumption. Surge pricing needs something to price, so reservations now carry a fee. I've also assumed the following, and you should correct any that are wrong:

- ▶ The fee is a flat base fee times a surge multiplier. [...]
- ▶ Loyalty points can be redeemed against the fee.

Reserve — the new requirements (excerpt) I

Surge pricing

- ▶ **REQ-SUR-2:** The multiplier is capped (default $3.0\times$) and is never below $1.0\times$.
- ▶ **REQ-SUR-3:** The rider sees the price and must confirm it. The confirmed price is locked for that reservation, even if surge changes during the 15 minutes.

Loyalty points

- ▶ **REQ-LOY-2:** Cancelled and expired reservations earn nothing. Expiry may carry a penalty (configurable, default none).

Fraud detection

- ▶ **REQ-FRD-2:** Rules include:
 - ▶ reserve/cancel churn (more than N in an hour);
 - ▶ many accounts on one device or payment method;
 - ▶ hoarding (repeated expiry without unlocking);

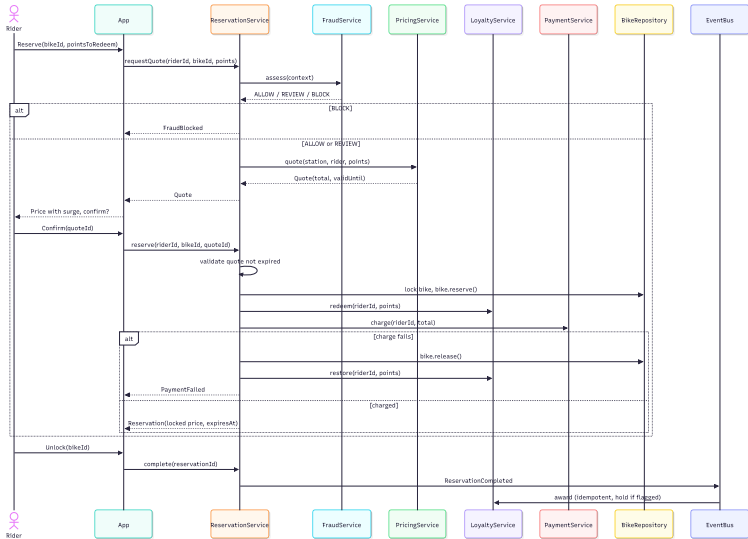
Reserve — the new requirements (excerpt) II

- ▶ reservations made from a location far from the station.
- ▶ **REQ-FRD-3:** BLOCK rejects the request. REVIEW allows it but flags it, and it earns no points until cleared.

[...]

Fraud tuning. The thresholds (N churn events, shared-device count) are placeholders. They should come from config and, ideally, be shadow-tested (log-only) before they start blocking real riders.

Reserve — the sequence



Reserve — the tests (excerpt)

```
@Test
void surgeMultiplierIsCappedAtThree() {
    surge.setDemandSupply("S1", 10, 1);           // raw ratio 10.0
    Quote q = pricing.quote("S1", "rider1", 0);

    assertEquals(3.0, q.multiplier());
    assertEquals(baseFee.times(3), q.total());
}
[...]
```

```
@Test
void expiredAndCancelledReservationsEarnNothing() {
    Reservation r = service.reserve("rider1", "B1", quoteId);
    clock.advance(Duration.ofMinutes(16));
    service.expireDue();

    assertEquals(0, loyalty.balance("rider1"));
}
[...]
```

```
@Test
void excessiveReserveCancelChurnIsBlocked() {
    for (int i = 0; i < 6; i++) history.recordCancel("rider1", clock.instant());

    FraudDecision d = fraud.assess(ctx("rider1"));

    assertEquals(FraudOutcome.BLOCK, d.outcome());
    assertTrue(d.reasons().contains("ChurnRule"));
}
```